Architecting a High-Performance, Local-First Browser Baseball Simulation Engine

The Paradigm of Client-Side Sports Simulation

The engineering of a fully self-contained, browser-based sports franchise simulation represents a unique intersection of advanced web architecture, mathematical modeling, and highly optimized data management. Operating entirely within the client environment demands solutions that entirely bypass the traditional client-server request lifecycle. Without a backend to handle heavy computational loads, manage relational databases, or orchestrate game states, the browser must independently execute database management, stochastic mathematical modeling, deterministic game logic, and high-performance user interface rendering.[1] The core challenge lies in building an application with the statistical depth of an established desktop application like Out of the Park Baseball, while maintaining the lightweight, accessible, and instantaneous responsiveness characterized by the ZenGM suite of browser games.[3]

Pioneering projects in this domain have established a proven blueprint for "local-first" gaming. These applications are built entirely in client-side JavaScript and TypeScript, leveraging IndexedDB for persistent storage and background threading for non-blocking simulation loops.[2] For a baseball dynasty simulation, the engine must manage the unique, highly discrete nature of baseball play-by-play mechanics, store decades of deep, multi-tiered statistical data, and render complex analytical dashboards without degrading frame rates or exhausting browser memory.[5]

This comprehensive intelligence report synthesizes proven open-source strategies, optimal state management methodologies, robust mathematical modeling frameworks, and high-efficiency rendering techniques. The objective is to provide a direct, actionable technical blueprint to construct a standalone React application that avoids reinventing foundational architectures.

Analysis of Open Source Baseball Simulation Engines

The open-source ecosystem provides several foundational approaches to baseball game simulation, ranging from highly procedural scripts to complex, object-oriented, multi-threaded applications. Analyzing these repositories reveals the specific patterns that succeed and the architectural anti-patterns that must be avoided when scaling to a multi-season dynasty format.

The most rudimentary approaches, such as the sernadesigns/baseball-game repository, utilize procedural functional recursion to manage the flow of the game. In this architecture, game initialization triggers a function that sets the inning state, which subsequently calls a top-of-the-inning function. This function loops a discrete at-bat routine until three outs are recorded, resetting the out counter and passing control to a bottom-of-the-inning function.[5] Within the at-bat routine, a pitch-throwing function selects a random pitch type from an array and passes the result to a batting outcome function, which recursively calls itself based on strike and ball counts.[5] While this procedural recursion is easy to read and implement, it is structurally fragile. It tightly couples the game state to the call stack, making it highly susceptible to race conditions and memory leaks when scaling up to include complex rules like double plays, balks, base-stealing, and mid-inning pitching substitutions.

Conversely, the snoozle-software/monte-carlo-mlb repository demonstrates the value of statistical abstraction over physics-based simulation. Written in Java, this engine eschews the granular modeling of rare events—such as pitcher pick-offs, balks, and fielding errors—arguing that over the course of thousands of iterations in a Monte Carlo simulation, these rare events dissolve into statistical noise.[7] The engine relies heavily on real player data, running continuous simulations to calculate win probabilities and standard deviations of combined scores based strictly on projected performance metrics like On-Base Percentage (OBP) and Slugging Percentage (SLG).[7] For a browser-based franchise mode that must simulate thousands of games per season in seconds, adopting this philosophy of statistical abstraction over granular physical modeling is computationally essential.

Further architectural maturity is observed in the kuhe/Angular-Baseball repository. While the front-end is bound to an older Angular framework, the core baseball logic engine was deliberately decoupled and written in TypeScript.[8] The developer organized the codebase to strictly separate the object-oriented baseball model from the mathematical services and the front-end rendering logic.[8] This repository also includes an automated continuous integration test that simulates a ten-year career of a random player purely to verify that the resulting statistical outputs mirror realistic historical averages.[8] This pattern of isolating the core engine from the user interface and utilizing automated statistical validation is a critical requirement for a React-based implementation.

Finally, the scottwillson/play-by-play engine introduces the most sophisticated architectural paradigm by modeling the game as a series of finite states and Markov chains. Rather than relying on nested loops, the game is treated as an immutable chain of possession states. Each state possesses a set number of valid transitions to other states, with each transition governed by a specific probability.[9] This approach eliminates the brittle nature of procedural recursion and forms the ideal mathematical and logical foundation for a highly performant, deterministic baseball simulation in a modern JavaScript environment.

The ZenGM Architectural Blueprint

The ZenGM suite, created by Jeremy Scheff, serves as the definitive reference architecture for serverless, browser-based sports simulations. A historical analysis of the ZenGM codebase reveals a deliberate evolution from traditional server-side technologies to a highly optimized, local-first JavaScript paradigm. In its infancy, the project relied on Python, SQLite, and GTK, eventually migrating to a web-based JavaScript application relying on jQuery, and culminating in a modern architecture utilizing TypeScript, React, and IndexedDB.[10]

The Shared Worker and Caching Paradigm

A defining architectural decision in the ZenGM engine is the total exile of the core game code from the browser's main thread. The simulation engine runs entirely inside a SharedWorker, with a standard Web Worker fallback for browsers that lack shared thread support.[2] The React front-end operates strictly as a "dumb" presentation layer. User interactions trigger discrete events that are passed to the worker thread via an internal messaging protocol. The worker performs the necessary calculations, updates the database, and returns the mutated state back to the UI.[1]

However, relying solely on IndexedDB for real-time data retrieval during a heavy simulation loop introduces severe disk read/write bottlenecks. To circumvent this, the ZenGM architecture implements a sophisticated, in-memory caching layer (src/worker/db/Cache.ts) that sits directly on top of the IndexedDB interface.[1] During the simulation of a season, all active players, team rosters, and current statistical accumulations are loaded into this volatile memory cache. The simulation engine executes its thousands of calculations against the cache, drastically reducing disk I/O overhead. IndexedDB is only queried directly for uncommon, historical requests, such as the user navigating to a retired player's profile page to view statistics from decades prior.[4]

Because the cache returns mutable values, the architecture demands strict discipline from the developer. If a value is purposely mutated during a simulation step, the engine must manually serialize the change back to the cache, and subsequently flush the cache to IndexedDB at safe intervals to ensure data persistence.[1]

Player Development and Determinism

In a dynasty simulation, the logic governing how players improve or decline over multiple seasons is critical to maintaining a realistic universe. Early iterations of the ZenGM development algorithm suffered from extreme volatility, where low-rated rookies could miraculously transform into league Most Valuable Players within a single off-season, or where aging players lost their underlying basketball skills rather than just their physical athleticism.[13]

To rectify this, the engine decoupled physical ratings from intelligence ratings. The introduction of specific metrics like Offensive IQ and Defensive IQ ensured that as players aged and their physical attributes naturally deteriorated, their high intelligence ratings allowed them to maintain effectiveness, accurately mirroring real-world veteran aging curves.[13] Furthermore, the engine restricted the velocity of development. Progression was tuned to be more gradual

throughout a player's teens and twenties, preventing the unrealistic morphological changes of player archetypes.[14]

When integrating historical, real-world players into the simulation, the engine faced the challenge of balancing historical accuracy with the randomness required for a compelling game. If historical ratings perfectly matched real life, the simulation became entirely predictable. The implementation of "Real Player Development Determinism" solved this by applying a weighted average between the player's real-world progression data and the engine's randomized progression algorithm.[15] This ensures that while a historical superstar has a high probability of becoming a great player, there remains a stochastic chance that their career diverges due to injury or poor development, preserving the sandbox nature of the simulation.

Finite State Machines for Baseball Logic

As evidenced by the fragility of procedural loops in earlier open-source examples, modeling the complex, hierarchical rules of a baseball game requires strict state governance. A Finite State Machine (FSM) is an architectural pattern that restricts a system to existing in exactly one defined state at any given time, strictly defining the valid transitions between those states based on discrete events.[17]

In the context of a React application, embedding complex conditional logic directly into component event handlers or useEffect hooks rapidly leads to unmaintainable code and state inconsistencies. The XState library provides a robust, visual, and highly scalable framework for implementing statecharts in modern JavaScript.[19] By utilizing XState, the baseball simulation logic is completely abstracted away from the React component tree. The state machine acts as an independent brain, exposing its current context and valid transitions to the UI purely for rendering purposes.[20]

A formalized baseball state machine encapsulates the game flow cleanly. The initial state is defined as the pre-game environment. Upon receiving a specific initialization event, the machine transitions into an active half-inning state.[18] The primary operational state within an inning is the at-bat. When the machine is in the "Batter Up" state, it listens for pitch events. The resulting state transitions are dictated by the outcome of those pitches. A ball event loops back to the "Batter Up" state while incrementing an internal context variable for the ball count. A strike event operates similarly. A contact event transitions the machine into a "Live Ball" state, where it awaits secondary events such as defensive fielding outcomes or baserunner advancements.[18]

Current State Node	Trigger Event	Resulting State	Contextual Guard /

		Transition	**Condition**
Pre_Game	INITIALIZE_GAME	Top_Inning	Inning = 1, Outs = 0
Batter_At_Plate	PITCH_BALL	Update_Count	Pitch result is Ball. Count < 3
Batter_At_Plate	PITCH_STRIKE	Update_Count	Pitch result is Strike. Count < 2
Batter_At_Plate	BALL_FOUR	Resolve_Walk	Ball count reaches 4
Batter_At_Plate	STRIKE_THREE	Resolve_Out	Strike count reaches 3
Batter_At_Plate	BALL_IN_PLAY	Ball_In_Play	Contact made by batter
Ball_In_Play	DEFENSIVE_OUT	Resolve_Out	Fielder successfully records out
Ball_In_Play	SAFE_HIT	Resolve_Hit	Batter reaches base safely
Resolve_Out	CHECK_INNING	Next_Batter	Total inning outs < 3
Resolve_Out	CHECK_INNING	Switch_Sides	Total inning outs = 3

Switch_Sides	CHECK_GAME_END	Post_Game	Inning >= 9, Home team leads

By strictly defining these transitions, the XState machine ensures that impossible scenarios—such as a batter striking out on a two-strike count, or a half-inning continuing after three outs—are mathematically impossible.[17] Furthermore, the entire state machine configuration can be serialized as JSON, allowing the simulation engine to pause a game, save the exact state context to the database, and seamlessly resume execution upon reload.[22]

Mathematical Modeling via Markov Chains

Beneath the overarching governance of the FSM, the actual determination of baseball events is most accurately achieved through Markov Chains. A Markov Chain is a stochastic mathematical model describing a sequence of possible events where the probability of each event depends only on the state attained in the previous event.[23] This "memoryless" property aligns perfectly with the mechanics of baseball: the probability of a team scoring runs from a bases-loaded, one-out scenario is identical regardless of whether the bases were loaded via three consecutive singles or a walk, an error, and a hit-by-pitch.[24]

At any given moment during a plate appearance, the game exists in one of 24 distinct base-out states. These states are derived from the eight possible configurations of base runners (bases empty, runner on first, runner on second, runner on third, runners on first and second, runners on first and third, runners on second and third, and bases loaded) combined with the three possible out configurations (zero, one, or two outs).[25] A 25th state serves as the "absorbing state," representing the third out and the termination of the half-inning.[27]

The core of the simulation engine relies on a 24×24 transition matrix, commonly denoted as $\mathbf{P}$. Each entry within this square, right-stochastic matrix represents the exact probability of the game transitioning from one specific base-out state to another during a single plate appearance.[24]

The transition matrix is logically structured as a series of block submatrices representing the constraints of the sport:

- Submatrices where the arrangement of base runners changes, but the out count remains static (e.g., advancing on a base hit).
- Submatrices where base runners may advance, but the out count increases by exactly one (e.g., a sacrifice bunt or a standard ground out).
- Submatrices where the out count increases by exactly two (e.g., a double play).
- Submatrices mapping the transition directly to the terminal three-out absorbing state.

- Zero-probability blocks representing impossible transitions, such as any event that would result in a decrease in the current number of outs.[28]

To populate this transition matrix with realistic probabilities, the simulation engine relies on massive datasets compiled by organizations such as Retrosheet. By parsing decades of historical play-by-play data, precise empirical probabilities are calculated for every possible transition.[29] For example, historical data precisely dictates the probability that a runner on first will advance to third on a single to right field, allowing the matrix to reflect true-to-life baserunning outcomes without requiring complex physics engines.[24]

While a static transition matrix based on league averages provides a baseline simulation, a compelling dynasty game requires individualized player impact. The engine achieves this by dynamically modifying the matrix probabilities based on the specific attributes of the current batter and pitcher. If a batter possesses exceptional power ratings, the transition probabilities leading to a "bases empty" state (representing a home run) are mathematically inflated for that specific at-bat.[31] Conversely, if a pitcher has an elite strikeout rating, transitions that increment the out count without advancing runners are increased. By continually multiplying the current state vector by these dynamically adjusted transition matrices, the engine rapidly and accurately simulates the accumulation of runs, hits, and outs over the course of an entire season.[23]

Reproducible Determinism via Seeded PRNGs

A paramount requirement for a browser-based simulation game is determinism. If a user closes their browser mid-season and subsequently reloads their save file, the game must proceed exactly as it would have prior to the interruption. This prevents "save-scumming"—where a user repeatedly reloads the game to achieve a favorable outcome—and ensures that the simulation environment is stable and testable.[32]

The native JavaScript Math.random() function is fundamentally incapable of providing this determinism. The ECMAScript specification does not mandate a specific underlying algorithm for Math.random(), nor does it provide any mechanism for supplying an initial seed value. As a result, different browsers implement different generators, and every invocation produces a completely independent, unpredictable sequence of numbers.[33]

To achieve exact reproducibility across user sessions and browser environments, the application must implement a Seeded Pseudo-Random Number Generator (PRNG). A PRNG utilizes an initial seed value to establish its internal state. Every subsequent request for a random number applies a deterministic mathematical operation to that state, producing the requested output and updating the state for the next call.[35] Consequently, providing the exact same seed value will invariably produce the exact same sequence of pseudo-random numbers.[35]

The two dominant libraries in the JavaScript ecosystem for seeded generation are seedrandom and pure-rand.

The seedrandom library has a long history and offers the ability to globally override the native Math.random() function.[33] While this global mutation is convenient for retrofitting older codebases that rely heavily on native math calls, it introduces significant side effects in modern, modular applications. Furthermore, performance benchmarks reveal that the algorithms utilized by seedrandom execute at a speed penalty, running roughly 1.1x to 1.8x slower than native math operations.[36]

For a high-performance simulation engine requiring millions of continuous random calculations per second, the pure-rand library is the demonstrably superior choice. Written natively in TypeScript, pure-rand avoids global mutation entirely. Instead, it generates a discrete, immutable generator instance that is passed functionally through the simulation logic.[37]

Feature Category	**seedrandom Library**	**pure-rand Library**
Language Ecosystem	JavaScript	TypeScript (Native)
Core Algorithms	ARC4, Alea, Xorwow	Xoroshiro128+, Mersenne Twister
Global Mutation	Common (overrides Math.random())	None (Pure functional instances)
Execution Speed	1.1x - 1.8x slower than native methods	Highly optimized (Xoroshiro128+ achieves ~24.5ms mean time for bulk generation)
Primary Use Case	Legacy codebase retrofitting	High-performance property testing, complex game simulations

Data derived from repository benchmarks and documentation.[36]

The pure-rand library offers exceptional performance, particularly when utilizing the xoroshiro128plus algorithm.[38] Most importantly, it allows the developer to easily extract the precise internal state of the generator array.[37] By serializing this array and committing it to the database alongside the user's save state, the engine can deserialize the array upon reload and instantly resume the exact mathematical sequence of the simulation, ensuring flawless continuity.[37]

Threading Architecture: Single-File Web Workers

JavaScript operates within a single-threaded event loop environment. Executing a computationally heavy process, such as simulating the outcome of thousands of baseball games across an entire season, will completely monopolize the main thread. This monopolization prevents the browser from processing user inputs, executing layout recalculations, or rendering DOM updates, resulting in an entirely frozen UI and potential browser-level crash warnings.[40]

To preserve the responsiveness of the React interface, all intensive mathematical modeling and database interactions must be explicitly offloaded to a Web Worker. A Web Worker executes JavaScript code in a fully isolated background thread, safely insulated from the main execution context.[40] This pattern allows the main thread to remain entirely dedicated to rendering the React component tree and responding to user events, while the worker silently processes the simulation loop.[40]

Achieving a Zero-Backend, Single-File Deployment

The specific requirement to deliver the application as a fully self-contained, single-file deployment complicates standard Web Worker instantiation. Typically, a Web Worker requires the browser to make an external network request to fetch a separate .js file from the server (e.g., new Worker('worker.js')). If the architecture dictates a single HTML/JS bundle with no external dependencies, this traditional network fetch will fail.[43]

The architectural solution involves dynamically constructing the Web Worker in memory by converting the worker's logic into a Blob object. The process operates through a specific sequence of APIs:

1. The worker's logic is written as a standard JavaScript module.
2. During the build process, a bundler isolates and transpiles the worker code into a continuous string.[44]
3. At runtime, the application instantiates a new Blob object, passing the stringified code as an array and defining the MIME type as application/javascript.[45]
4. The URL.createObjectURL(blob) method converts this in-memory Blob into a virtual,

browser-internal URL.[45]

5. The application invokes new Worker(virtualUrl), effectively spawning the background thread without requiring any external file requests.[46]

Modern build tools like Vite natively abstract this complex process. Developers can simply import the worker file and append the ?worker&inline query parameter. During the build step, Vite automatically compiles the worker, converts it into a base64 string, and inlines it directly into the primary application bundle, seamlessly satisfying the single-file deployment mandate while maintaining developer ergonomics.[47]

Bridging the Thread Gap with Comlink

Because Web Workers operate in an isolated scope, they lack direct access to the DOM or the React context.[40] Communication between the main thread and the worker relies exclusively on the postMessage API, which is highly asynchronous and notoriously cumbersome to manage, often leading to deeply nested, fragile event listeners.[40]

To establish a robust architecture, the application should integrate Comlink, an RPC (Remote Procedure Call) library developed by Google Chrome Labs. Comlink elegantly abstracts the postMessage protocol by wrapping the Web Worker in an ES6 Proxy.[49] This allows the React application to invoke functions residing inside the isolated worker thread exactly as if they were local asynchronous functions on the main thread, drastically simplifying the codebase.[49]

When the worker thread completes the simulation of a season, it must pass a massive payload of statistical data back to the React UI for rendering. By default, the browser utilizes the Structured Clone algorithm to copy data across the thread boundary, which is computationally expensive and doubles the memory footprint.[50] To prevent memory spikes, the architecture must utilize "Transferable Objects." By transferring the underlying ArrayBuffer of the data payload rather than copying it, the engine achieves zero-copy memory transfers, shifting ownership of the memory block directly from the worker to the main thread with near-zero latency.[42]

Persistent Data Layer: IndexedDB and Schema Design

Client-side data persistence is an existential requirement for a browser-based franchise simulation. The engine must possess a database capable of storing complex rosters, granular historical player statistics, full game logs, and team standings spanning dozens or even hundreds of simulated seasons.[52]

The Inadequacy of LocalStorage

While localStorage provides a simplistic, widely understood key-value API, it is fundamentally incompatible with the demands of a high-fidelity, long-term sports simulation due to three

severe limitations:

1. **Strict Capacity Limits:** localStorage is capped at a rigid 5MB to 10MB limit across nearly all browsers. A single season of detailed baseball play-by-play logs and advanced statistics will instantly overwhelm this quota, causing catastrophic application failure.[51]
2. **Synchronous Execution:** localStorage is entirely synchronous. Executing a read or write operation on a multi-megabyte JSON string blocks the main execution thread completely, freezing the UI and negating the performance benefits achieved by utilizing Web Workers.[54]
3. **Data Serialization Overhead:** It is incapable of storing native JavaScript objects or binary data, supporting only strings. This necessitates the use of JSON.stringify() and JSON.parse() for every database interaction, introducing massive CPU overhead for large datasets.[56]

The Power of IndexedDB and Dexie.js

IndexedDB is the mandatory storage mechanism for this architecture. It is a highly capable, asynchronous, client-side NoSQL database designed explicitly for storing large volumes of structured data.[54] It handles megabytes to gigabytes of information and supports the storage of complex data types, arrays, and objects natively, bypassing serialization overhead entirely.[52] Crucially, IndexedDB can be accessed directly from within the Web Worker thread. This allows the simulation loop to execute mathematical models, update records, and commit changes to the disk in the background without ever interrupting the React UI thread.[57]

Because the native IndexedDB API is event-driven and notoriously verbose, interacting with it directly results in deeply complex callback chains.[55] The application should wrap the native API using a Promise-based abstraction layer, specifically Dexie.js. Dexie provides powerful compound indexing, robust transactional integrity, and highly efficient bulk-write methods (bulkAdd, bulkPut) that are essential for committing thousands of generated box scores to the disk simultaneously at the conclusion of a simulated day.[55]

Optimized JSON Schema Design for Multi-Season Sims

Storing extensive historical data requires strict adherence to database normalization principles to maintain long-term query performance. A common anti-pattern in document-based NoSQL design—and an early architectural mistake explicitly documented in the development of ZenGM—is storing an entire entity's history embedded inside a single monolithic object.[6] If a team's statistical output is stored as an ever-expanding array appended to the main Team document, the document size grows infinitely with each passing season. Because IndexedDB cannot execute partial writes, updating the current season's stats requires reading the entire multi-decade history into memory, appending the new data, and rewriting the massive object back to the disk. This results in cascading performance degradation as the user progresses deeper into the simulation.[6]

To ensure the database scales indefinitely without performance penalties, the schema must be fully normalized, separating static entities from highly volatile, season-specific statistical records.[6]

An optimized IndexedDB JSON schema, heavily inspired by the SportsJS and SportsML interchange standards [63], requires the following discrete object stores:

Object Store Definition	**Primary Key**	**Required Indexes**	**Architectural Purpose**
players	playerId	teamId, isActive	Stores static biographical data (Name, DOB) and current overall physical ratings.
teams	teamId	conferenceId, divisionId	Stores static franchise data (City, Mascot, Stadium, Colors).
player_seasons	[playerId+season]	season, teamId	Tracks a player's volatile statistical accumulation (Hits, ERA, WAR) for a specific year.
team_seasons	[teamId+season]	season	Tracks a team's win/loss record, payroll, and cumulative stats for a specific year.
games	gameId	season, homeTeam, awayTeam	High-level metadata for a specific match (Date, final score,

			attendance).
box_scores	gameId	season	Highly granular play-by-play logs and individual game performances.

By architecting the schema to separate player_seasons from the core players object, the simulation engine achieves $O(1)$ memory consumption regarding historical depth. The Web Worker only needs to load the current active season index into its volatile cache to execute the simulation.[4] The vast accumulation of historical data remains dormant on the disk, queried via Dexie only when the user explicitly navigates to a player's career history dashboard.[6]

High-Performance UI: Rendering the Statistics Grid

The visualization of complex sports data presents the final, most significant challenge to React's rendering engine. A comprehensive player roster or historical statistics table displaying 1,000 players across 30 statistical columns (AB, R, H, HR, RBI, AVG, OBP, SLG, OPS, wOBA, WAR, etc.) results in an enormous DOM footprint.

Rendering a table of that magnitude simultaneously creates upwards of 30,000 discrete HTML nodes. Web browsers begin to experience severe layout thrashing, style recalculation delays, and plummeting frame rates when the DOM exceeds a recommended depth of roughly 1,500 nodes.[64] Forcing React to render and reconcile a massive statistical grid without architectural intervention inevitably leads to exorbitant memory heap expansion, unresponsive interactions, and a severely degraded user experience.[65]

The Necessity of DOM Virtualization

To circumvent the inherent limits of the browser DOM, the application must aggressively utilize "virtualization" (also referred to as "windowing"). Virtualization is a rendering technique that mathematically calculates the user's current scroll position and dynamically renders only the specific rows that are currently visible within the viewport (along with a small off-screen buffer to prevent visual tearing during rapid scrolling).[66]

As the user scrolls down the statistical grid, the virtualization library seamlessly recycles the existing DOM nodes, swapping out the underlying data payload rather than creating new elements or destroying old ones. This ensures that the total number of DOM elements remains strictly capped and minimal, regardless of whether the dataset contains 100 rows or 100,000

rows, preserving smooth 60 FPS performance.[65]

Choosing the Optimal Virtualization Library: react-window vs. react-virtuoso

Selecting the correct virtualization library dictates the flexibility, accessibility, and maintainability of the application's user interface.[69]

Historically, react-virtualized was the industry standard. However, it is now considered deprecated and carries a highly bloated package size of approximately 33.5 KB.[70] Its original author subsequently created react-window as a complete rewrite, providing a minimalist, highly performant core utilizing only 2 KB of space.[70] While react-window is incredibly fast, it achieves this speed through structural rigidity. It requires the developer to strictly hardcode absolute pixel heights for every row, and it does not natively support complex table layouts or crucial features like sticky headers without heavy, custom modifications.[70] Attempting to engineer a sticky header into react-window often results in excessive component re-renders, breaking the very performance benefits the library was installed to provide.[72]

For the specific demands of a complex baseball dashboard, **react-virtuoso** emerges as the superior architectural choice.[73]

While carrying a slightly larger footprint than react-window, react-virtuoso was engineered specifically to handle the modern requirements of complex data grids without demanding rigid constraints from the developer [70]:

1. **Dynamic Sizing:** It inherently utilizes the browser's ResizeObserver API to calculate the height of items dynamically on the fly, entirely eliminating the need for the developer to calculate or hardcode exact row dimensions.[70]
2. **Native Table Support:** Unlike basic list virtualizers that force the use of absolutely positioned elements, react-virtuoso provides a dedicated <TableVirtuoso> component. This component seamlessly renders semantic HTML <table>, <thead>, <tbody>, and <tr> tags, ensuring that the statistics grid remains fully accessible to screen readers and inherently responsive to CSS styling.[73]
3. **Sticky Headers and Fixed Columns:** Crucial for digesting massive arrays of sports statistics, react-virtuoso inherently supports frozen headers. This allows the user to scroll vertically through thousands of players while keeping the statistical column labels pinned to the top of the viewport. Furthermore, it supports fixed columns, allowing the user to scroll horizontally across dozens of stat categories while keeping the vital Player Name column pinned securely to the left axis.[76]

By funneling the massive datasets generated by the Web Worker simulation and stored in IndexedDB through the TableVirtuoso component, the application guarantees that the user can instantly interact with and analyze decades of generated league history with frictionless

performance.[65]

Strategic Synthesis

Constructing a highly advanced, browser-based baseball franchise simulation in a single file without a backend requires the precise orchestration of modern browser APIs, rigorous architectural discipline, and optimal library selection.

To achieve maximum performance and mathematical reliability, the architecture must abandon brittle procedural recursion and rely on **XState** to govern the strict, immutable transitions of inning logic. This state machine must be fed by **Markov Chain probability matrices**, derived from empirical Retrosheet data, to accurately simulate the statistical realities of the sport. The inherent randomness dictating these matrix transitions must be supplied by the **pure-rand** library, utilizing the highly optimized Xoroshiro128+ algorithm to guarantee deterministic, exactly reproducible outcomes across all user sessions.

The entire computational workload of the simulation must be explicitly exiled from the main thread to preserve UI rendering speed. It must be isolated within a **Web Worker**, uniquely initialized dynamically via an in-memory **Blob URL** to satisfy the single-file deployment constraint, and communicating cleanly with the React frontend via **Comlink's** proxy interface. The state of the simulated universe must be continuously preserved in an **IndexedDB** database, utilizing **Dexie.js** and a heavily normalized, multi-store JSON schema to prevent read/write bottlenecks and memory bloat as the user simulates deep into future decades. Finally, the vast oceans of data produced by the engine must be passed back to the React UI and rendered exclusively through the **react-virtuoso** library, ensuring the browser DOM remains structurally light, semantically accessible, and impeccably responsive.

By strictly adhering to this comprehensive architectural blueprint, the developer can successfully engineer an expansive, local-first simulation experience that operates entirely within the boundaries of a single client-side deployment.

Works cited

1. zengm-games/zengm-legacy: Some old video games from ... - GitHub, accessed February 24, 2026, https://github.com/zengm-games/zengm-legacy
2. nicidob/gm-games: Single-player and management simulation games, made entirely in client-side JavaScript. - GitHub, accessed February 24, 2026, https://github.com/nicidob/gm-games
3. Blog « ZenGM, accessed February 24, 2026, https://zengm.com/blog/15/
4. Basketball GM (and other ZenGM games) are single-player sports management simulation games, made entirely in client-side JavaScript. - GitHub, accessed February 24, 2026, https://github.com/zengm-games/zengm
5. A simple baseball simulation game in JavaScript, bundled with Webpack. - GitHub, accessed February 24, 2026,

https://github.com/sernadesigns/baseball-game
6. Game simulation performance improvements « Blog « - ZenGM, accessed February 24, 2026, https://zengm.com/blog/2016/04/game-simulation-performance-improvements/
7. snoozle-software/monte-carlo-mlb: Major league baseball game predictor that uses a monte carlo technique to simulate games - GitHub, accessed February 24, 2026, https://github.com/snoozle-software/monte-carlo-mlb
8. kuhe/Angular-Baseball: The great American pastime in JavaScript (versioned bundle files for GitHub hosting), accessed February 24, 2026, https://github.com/kuhe/Angular-Baseball
9. scottwillson/play-by-play: Stats-based basketball simulation - GitHub, accessed February 24, 2026, https://github.com/scottwillson/play-by-play
10. Porting Basketball GM to TypeScript « Blog « - ZenGM, accessed February 24, 2026, https://zengm.com/blog/2020/01/porting-to-typescript/
11. Nostalgia from my changelog, and how to play 2013 BBGM today « Blog « - ZenGM, accessed February 24, 2026, https://zengm.com/blog/2020/10/nostalgia/
12. Basketball GM 4.0 technical details - caching, Shared Workers, IndexedDB/Promise interactions, Safari being a tease, McDonald's, and more « Blog « - ZenGM, accessed February 24, 2026, https://zengm.com/blog/2017/04/basketball-gm-4-0-technical-details/
13. Player ratings and development beta! « Blog « - ZenGM, accessed February 24, 2026, https://zengm.com/blog/2018/02/player-ratings-and-development-beta/
14. Revamped player development algorithm « Blog « - ZenGM, accessed February 24, 2026, https://zengm.com/blog/2014/09/revamped-player-development-algorithm/
15. Posts tagged "Player Development" « Blog « - ZenGM, accessed February 24, 2026, https://zengm.com/blog/tag/player-development/
16. Rescaled real player development determinism « Blog « - ZenGM, accessed February 24, 2026, https://zengm.com/blog/2021/03/rescaled-real-player-development-determinism/
17. State Machines in JavaScript with XState - Angular.love, accessed February 24, 2026, https://angular.love/state-machines-in-javascript-with-xstate/
18. Baseball is a State Machine - Donovan LaDuke - Developer, accessed February 24, 2026, https://dladukedev.com/articles/015_state_machines/
19. statelyai/xstate: Actor-based state management & orchestration for complex app logic. - GitHub, accessed February 24, 2026, https://github.com/statelyai/xstate
20. Using State Machines with React and XState | by Robert Vogt | smartive - Medium, accessed February 24, 2026, https://medium.com/smartive/using-state-machines-with-react-and-xstate-bb87c3bc215e
21. State Machines in React - Mastery Games, accessed February 24, 2026, https://mastery.games/post/state-machines-in-react/
22. How to use finite state machines in React? : r/javascript - Reddit, accessed February 24, 2026,

https://www.reddit.com/r/javascript/comments/opx5jc/how_to_use_finite_state_machines_in_react/
23. A MARKOV CHAIN MODEL FOR RUN PRODUCTION IN BASEBALL - Winthrop University, accessed February 24, 2026, http://faculty.winthrop.edu/polaskit/Spring13/Baseball.pdf
24. The Elegance of Markov Chains in Baseball | by Lucas Calestini | Sports Analytics | Medium, accessed February 24, 2026, https://medium.com/sports-analytics/the-elegance-of-markov-chains-in-baseball-f0e8e02e7ac4
25. analysis of a baseball simulation game using markov chains - Lehigh University, accessed February 24, 2026, https://www.lehigh.edu/~dmd1/apba3.pdf
26. Markov Chains and Baseball - Ryan's Repository of Random Reflections, accessed February 24, 2026, http://www.ryanhmckenna.com/2014/11/markov-chains-and-baseball.html
27. (PDF) A Markov Chain Approach to Baseball - ResearchGate, accessed February 24, 2026, https://www.researchgate.net/publication/238836772_A_Markov_Chain_Approach_to_Baseball
28. The Markov Chain Model of Baseball - statshacker, accessed February 24, 2026, http://statshacker.com/blog/2018/05/07/the-markov-chain-model-of-baseball/
29. A Markov Chain Model for Predicting Major League Baseball - The University of Texas at Austin, accessed February 24, 2026, https://repositories.lib.utexas.edu/bitstreams/88eb5af3-a824-487c-829e-46def421a869/download
30. Evaluating Traditional Lineups - Retrosheet, accessed February 24, 2026, https://www.retrosheet.org/Research/RuaneT/lineup_art.htm
31. A Markov Chain Approach to Baseball - NJIT, accessed February 24, 2026, https://web.njit.edu/~bukiet/Papers/ball.pdf
32. tc39/proposal-seeded-random: Proposal for a new SeededPRNG class that yields reproducible sequences of random numbers. - GitHub, accessed February 24, 2026, https://github.com/tc39/proposal-seeded-random
33. Seeding the random number generator in JavaScript - Stack Overflow, accessed February 24, 2026, https://stackoverflow.com/questions/521295/seeding-the-random-number-generator-in-javascript
34. Random Number Generation in JavaScript - Bocoup, accessed February 24, 2026, https://www.bocoup.com/blog/random-numbers
35. Reproducible randomness and its applications - DEV Community, accessed February 24, 2026, https://dev.to/krofdrakula/reproducible-randomness-and-its-applications-1ong
36. davidbau/seedrandom: seeded random number generator for Javascript - GitHub, accessed February 24, 2026, https://github.com/davidbau/seedrandom
37. Random | API Reference | fast-check | Property based testing framework, accessed February 24, 2026, https://fast-check.dev/api-reference/classes/Random.html

38. dubzzz/pure-rand: Pure random number generator written in TypeScript - GitHub, accessed February 24, 2026, https://github.com/dubzzz/pure-rand
39. Did any one implement web pages with random content? : r/sveltejs - Reddit, accessed February 24, 2026, https://www.reddit.com/r/sveltejs/comments/16gsppc/did_any_one_implement_web_pages_with_random/
40. Optimizing React Apps with Web Workers | by Frontend Highlights - Medium, accessed February 24, 2026, https://medium.com/@ignatovich.dm/optimizing-react-apps-with-web-workers-cb01b9d8f77c
41. Optimizing Long Tasks in React: Chunking and Web Workers for Better Performance, accessed February 24, 2026, https://javascript.plainenglish.io/optimizing-long-tasks-in-react-chunking-and-web-workers-for-better-performance-1d68ea5777b7
42. Web Workers Explained: Unlocking True Concurrency - DEV Community, accessed February 24, 2026, https://dev.to/mino/unlocking-performance-a-comprehensive-guide-to-web-workers-25i7
43. Implementing single file Web Components | by Tomasz Jakut | Content Uneditable | Medium, accessed February 24, 2026, https://medium.com/content-uneditable/implementing-single-file-web-components-22adeaa0cd17
44. create a web worker dynamically with an embedded worker blob. - GitHub Gist, accessed February 24, 2026, https://gist.github.com/dfkaye/14e5cc5dbe5bb38a9d80f25f54061c7f
45. Blob - Web APIs - MDN Web Docs, accessed February 24, 2026, https://developer.mozilla.org/en-US/docs/Web/API/Blob
46. Creating a web worker inside React - Stack Overflow, accessed February 24, 2026, https://stackoverflow.com/questions/47475360/creating-a-web-worker-inside-react
47. support `?inline` worker in constructors worker · Issue #7352 · vitejs/vite - GitHub, accessed February 24, 2026, https://github.com/vitejs/vite/issues/7352
48. Features | Vite, accessed February 24, 2026, https://vite.dev/guide/features
49. Web Workers in JavaScript: Multithreading Without the Pain | by Amir Hossein Hosseiny, accessed February 24, 2026, https://javascript.plainenglish.io/web-workers-in-javascript-multithreading-without-the-pain-c532378bc102
50. The State Of Web Workers In 2021 - Smashing Magazine, accessed February 24, 2026, https://www.smashingmagazine.com/2021/06/web-workers-2021/
51. Proposal that got me into GSoC 2025 (Google Summer of Code) | by Deeptanshu Sankhwar, accessed February 24, 2026, https://medium.com/@deeptanshu.sankhwar/proposal-that-got-me-intogsoc-2025-google-summer-of-code-71f082efd4c5
52. IndexedDB: A Comprehensive Overview for Frontend Developers - Medium,

accessed February 24, 2026, https://medium.com/@shashika.silva88/indexeddb-a-comprehensive-overview-for-frontend-developers-6b47a9f32e23
53. Browser Storage: A Comparative Analysis of IndexDB, Local Storage, and Session Storage - Browsee, accessed February 24, 2026, https://browsee.io/blog/unleashing-the-power-a-comparative-analysis-of-indexdb-local-storage-and-session-storage/
54. LocalStorage vs IndexedDB: Choosing the Right Solution for Your Web Application, accessed February 24, 2026, https://shiftasia.com/community/localstorage-vs-indexeddb-choosing-the-right-solution-for-your-web-application/
55. Any downsides to using indexedDB vs localStorage? : r/sveltejs - Reddit, accessed February 24, 2026, https://www.reddit.com/r/sveltejs/comments/15rj12h/any_downsides_to_using_indexeddb_vs_localstorage/
56. Development question (savefiles) : r/incremental_games - Reddit, accessed February 24, 2026, https://www.reddit.com/r/incremental_games/comments/ljoxs9/development_question_savefiles/
57. 9 differences between IndexedDB and LocalStorage - DEV Community, accessed February 24, 2026, https://dev.to/armstrong2035/9-differences-between-indexeddb-and-localstorage-30ai
58. IndexedDB vs. localStorage: When and Why to Use IndexedDB for Data Storage in Web Applications | by Sri Web | Medium, accessed February 24, 2026, https://medium.com/@sriweb/indexeddb-vs-localstorage-when-and-why-to-use-indexeddb-for-data-storage-in-web-applications-93a8a5a39eef
59. IndexedDB, WebSQL, LocalStorage – what blocks the DOM? - Nolan Lawson, accessed February 24, 2026, https://nolanlawson.com/2015/09/29/indexeddb-websql-localstorage-what-blocks-the-dom/
60. Angular and IndexedDB: A Perfect Pair for Progressive Web Apps | by zeeshan khan, accessed February 24, 2026, https://medium.com/@zeeshankhan8838/angular-and-indexeddb-a-perfect-pair-for-progressive-web-apps-315a39f49
61. TECH >> Testing performance of bulk writes in IndexedDB - Gautam Panickar SS - Medium, accessed February 24, 2026, https://gautampanickarss.medium.com/tech-testing-performance-of-bulk-writes-in-indexeddb-a463ed68ea60
62. How to design a sports statistics database for multiple sports? [closed] - Stack Overflow, accessed February 24, 2026, https://stackoverflow.com/questions/4404118/how-to-design-a-sports-statistics-database-for-multiple-sports
63. iptc/sportsjs-dev: Development of the IPTC sports-data-in-JSON format sportsJS - GitHub, accessed February 24, 2026,

https://github.com/iptc/sportsjs-dev
64. Optimizing React Table Rendering By 160x !!! - DEV Community, accessed February 24, 2026, https://dev.to/navneet7716/optimizing-react-table-rendering-by-160x--5g3c
65. How To Render Large Datasets In React without Killing Performance | Syncfusion Blogs, accessed February 24, 2026, https://www.syncfusion.com/blogs/post/render-large-datasets-in-react
66. Virtualization in React: Improving Performance for Large Lists | by Frontend Highlights, accessed February 24, 2026, https://medium.com/@ignatovich.dm/virtualization-in-react-improving-performance-for-large-lists-3df0800022ef
67. Infinite Scrolling Made Easy: react-window vs react-virtuso | by Stuthi Neal - Medium, accessed February 24, 2026, https://medium.com/@stuthineal/infinite-scrolling-made-easy-react-window-vs-react-virtuso-1fd786058a73
68. Optimizing Large Datasets with Virtualized Lists | by Eva Matova | Medium, accessed February 24, 2026, https://medium.com/@eva.matova6/optimizing-large-datasets-with-virtualized-lists-70920e10da54
69. Build Tables in React: Data Grid Performance Guide - Strapi, accessed February 24, 2026, https://strapi.io/blog/table-in-react-performance-guide
70. React Virtualization - react-window vs react-virtuoso - DEV Community, accessed February 24, 2026, https://dev.to/sanamumtaz/react-virtualization-react-window-vs-react-virtuoso-8g
71. react-virtualized vs. react-window - LogRocket Blog, accessed February 24, 2026, https://blog.logrocket.com/react-virtualized-vs-react-window/
72. Using react-window - is this good and performant code? : r/reactjs - Reddit, accessed February 24, 2026, https://www.reddit.com/r/reactjs/comments/1inzg8a/using_reactwindow_is_this_good_and_performant_code/
73. react-virtuoso - Codesandbox, accessed February 24, 2026, http://codesandbox.io/p/github/kirkbowe/react-virtuoso
74. Building High-Performance Virtual Lists With React Virtuoso - DhiWise, accessed February 24, 2026, https://www.dhiwise.com/post/mastering-react-virtuoso-comprehensive-guide-for-developers
75. Support grid layout? · Issue #13 · petyosi/react-virtuoso - GitHub, accessed February 24, 2026, https://github.com/petyosi/react-virtuoso/issues/13
76. Table Virtuoso Example with Fixed Headers, accessed February 24, 2026, https://virtuoso.dev/react-virtuoso/table-virtuoso/table-fixed-headers/
77. React Virtuoso, accessed February 24, 2026, https://virtuoso.dev/react-virtuoso/